

JAN-SEVA — Session Report

Citizen complaint flow: from browser-only prototype to a real Supabase backend.

Working window: 2 Sep 2026, 15:00 → 3 Sep 2026, 01:35 IST

1. Where we started (15:00)

A Vite + React single-page app with **every complaint living in the browser's localStorage**. The SQL and the serverless functions existed as files, but nothing had ever run:

- **No Supabase project existed.** Migrations 0001–0007 had never been applied anywhere.
- The three functions in `api/` (OTP send, OTP verify, AI screen) had **never executed in any runtime** — `vite dev` does not run them.
- No way to verify any of it. Test suite was 337 + 22 self-tests, all client-side.

2. What we built

Layer	Change
Database	Live project in ap-south-1 (Mumbai) . Migrations 0001–0010 applied: 24 tables, PostGIS, 2 storage buckets. Three new migrations written this session — 0008 (privileges), 0009 (atomic complaint creation), 0010 (atomic rate limiting).
Backend	POST /api/complaints/create and GET /api/complaints/:id (new). Identity attestation (new): HMAC-signed, 30-minute, server-issued proof of OTP verification. Edge runtime declared on all four functions.
Frontend	<code>submitReport</code> and <code>getById</code> both switched to the real API, mode-gated like the existing OTP service, with no localStorage fallback . No UI was changed — only the data layer underneath.
Verification	Five new suites, all run against the live database and all self-cleaning: <code>check:db</code> , <code>check:rls</code> , <code>check:complaint</code> , <code>check:http</code> , <code>check:track</code> .

The citizen loop is now real in both directions: a report filed through `/report` is written to Postgres in one transaction, gets a server-issued ticket like `JS-GWL-2026-000001`, and that same id is found again by `/track` through the public read endpoint.

3. Bugs found — and why testing mattered

Every one of these was invisible to code review. Each was caught by running the thing:

#	Bug	Why it hid
1	Grants were never issued, so all 24 tables refused every caller	The 403/42501 looked exactly like a failed migration. It proved the opposite — Postgres only says <i>permission denied</i> about a table that exists.
2	Admin could read other cities' complaints and staff	Latent. Only Gwalior was seeded, so it would have gone live the day Indore was added.
3	The error handler crashed inside its own catch block	<code>new URL()</code> on a relative path. The code written to stop stack traces leaking was producing the platform's raw error page.
4	All four API functions had the wrong runtime signature	Written Web-standard, run as Node. They had never executed anywhere, so nothing surfaced it.
5	Category vocabulary mismatch between app and database	Would have sent nearly every complaint to triage instead of a department.
6	Rate limiter failing open, and not atomic	Correct for sequential callers; bypassable by parallel ones — the only way an abusive script ever sends them. Burst of 25 against a limit of 15 gave zero refusals.
7	Split brain: writes to Postgres, reads from localStorage	A citizen would file successfully, get a real ticket number, then be told <i>not found</i> .

Two of these were bugs in work done *this session* — including an attestation scheme string containing a dot, which broke token parsing so that **nothing verified at all, while every forgery test passed** for exactly the wrong reason. Both were caught by adding a control check that proves the happy path works before any rejection is counted as evidence.

4. Test coverage now

Suite	Checks	What it proves
check:rls	74	Department isolation, IDOR blocked, anon reaches nothing
check:http	78	Trust boundary, attestation, atomic rate limiting
check:complaint	65	Server-issued IDs, routing, idempotency, rollback
check:track	52	Public projection, PII redaction, 48h expiry
check:db	—	Real read/write round trip
selftest + prod	359	Pre-existing client suites, still green

269 new automated checks against the live database, 0 failures. Every suite creates real rows and cleans up to zero.

5. Small things worth doing next

Quick wins, roughly in order of effort:

- **Commit this work.** 16 modified and 8 new files are still uncommitted.
- **Delete** `src/services/repositories/` — 371 lines that **nothing imports**. Dead on arrival; it will confuse the next reader.
- **Delete** `supabase/migrations/combined_all.sql` — a concatenation of 0001–0007 kept for pasting. It now reads like an eighth migration.
- **Pin the Vercel region to bom1.** The database is in Mumbai; Edge functions run globally, so a request from far away pays two long hops.
- **Add one** `npm run verify` that chains all five checks, so there is a single command before any deploy.
- **Document** `IDENTITY_SECRET` as required in `.env.example` — without it the rate limiter silently fails open.
- **Remove the stray Office temp file** (`~$SIH2026... .pptx`) and the 6 MB `.BACKUP.pptx` from the repository.

6. Still mock — stated plainly

Component	Status
Evidence / photo storage	Not wired. The tracking endpoint honestly reports <code>photoCount: 0</code> rather than inventing one. This is the natural next task.
Department & admin dashboards	Still read <code>localStorage</code> . Creation and citizen tracking are real; these are not.
OTP	Demo code 123456. Proven unusable in production by 22 self-tests.
AI screening	Real Gemini integration, deliberately inert — it refuses to send citizen photos until billing is acknowledged, because unpaid tiers may train on them.
Browser click-through	Not verified. The HTTP layer is proven end to end; nobody has clicked through the actual pages in a browser.

Overall: the citizen create-and-track loop is real and coherent. Not production-ready overall — evidence storage, the department and admin read paths, and a real browser pass are still outstanding.